

Database Management System Lab Project Report

HOSPITAL MANAGEMENT SYSTEM

Submitted By

Name: Sabbir Ahmed Emon

ID: 2023100000507

Course code: CSE384.6

Batch: 64

Submitted to

Faculty: Ms. Mahjabin Sultana, Lecturer, Southeast University

Submission date: 18.09.2026

1. Introduction

A Hospital Management System is a database system used to store and manage hospital information. It keeps information about patients, doctors, rooms, prescriptions, bills, and payments in an organized way.

In a hospital, many records are created every day. Managing all these records manually can be difficult and can cause mistakes or duplicate information. A database helps to store the information properly and makes it easier to find and manage records.

In this project, we created a Hospital Management System using Oracle Database. We created separate tables for doctors, patients, rooms, prescriptions, bills, and payments. These tables are connected using primary keys and foreign keys.

We used SQL to create tables, insert data, search for information, connect tables, and generate useful reports. We also used PL/SQL for concepts such as variables, IF-ELSE, arrays, loops, exception handling, procedures, functions, and cursors.

The main purpose of this project is to use the database concepts we learned in the laboratory and apply them to a real-life hospital management system.

2. Objectives

The main objectives of this project are:

1. To create a database for a Hospital Management System.
2. To store and manage doctor and patient information.
3. To keep records of patients and their assigned rooms.
4. To manage patient prescriptions and medicines.
5. To manage hospital bills and payment information.
6. To connect different tables using primary keys and foreign keys.
7. To retrieve useful information using SQL queries.

8. To use SQL concepts such as joins, operators, aggregate functions, subqueries, views, and set operators.
9. To use PL/SQL concepts such as %TYPE, %ROWTYPE, IF-ELSE, arrays, loops, exception handling, procedures, functions, and cursors.
10. To generate useful reports from the hospital database.
11. To understand how SQL and PL/SQL can be used in a real-life hospital system.

3. Problem Statement

Hospitals need to manage information about many patients, doctors, rooms, prescriptions, bills, and payments. If these records are maintained manually or in separate unorganized files, it becomes difficult to find accurate information quickly.

For example, hospital staff may need to answer questions such as:

- Which doctor is treating a particular patient?
- Which patients are assigned to a specific doctor?
- Which patients are staying in ICU rooms?
- What medicines have been prescribed to a patient?
- What is the total bill of a patient?
- Which bills are still pending?
- How many patients are being handled by each doctor?
- Which doctors have issued prescriptions?
- What is the payment status of a particular bill?

A properly designed database can solve these problems by keeping related information in connected tables.

Therefore, this project develops a Hospital Management System using Oracle SQL and PL/SQL to provide organized storage, retrieval, and processing of hospital information.

4. Database Requirements

The system contains the following main tables:

1. Doctor

2. Patient
3. Room
4. Prescription
5. Bill
6. Payment

The tables are connected using primary key and foreign key relationships.

5. ER Diagram

The Entity Relationship Diagram represents the major entities of the Hospital Management System and their relationships.

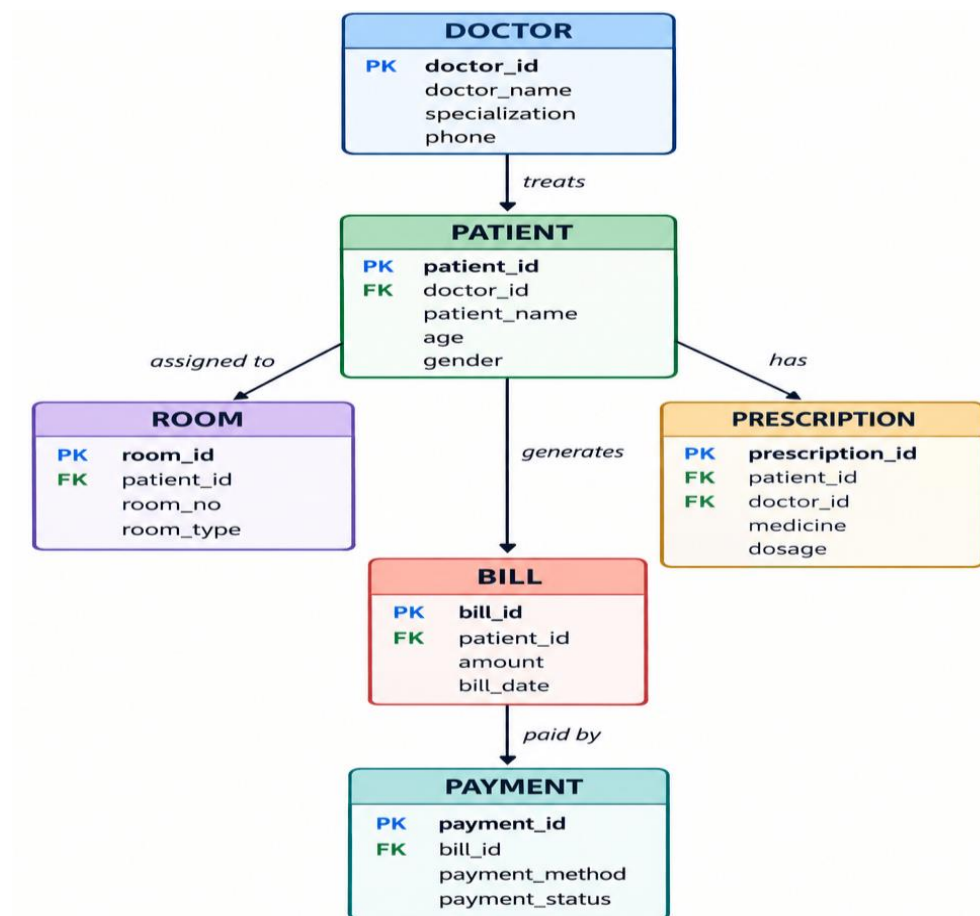
Main Entities

- **Doctor**
- **Patient**
- **Room**
- **Prescription**
- **Bill**
- **Payment**

Relationships

- A doctor can treat multiple patients.
- A patient is assigned to a doctor.
- A patient can have a room assignment.
- A patient can have prescriptions.
- A patient can have bills.
- A bill can have payment information.

ER Diagram :



6. Relational Schema

The relational schema of the system is as follows.

DOCTOR

DOCTOR(

doctor_id PK,

doctor_name,

```
    specialization,  
    phone  
)
```

PATIENT

```
PATIENT(  
    patient_id PK,  
    doctor_id FK,  
    patient_name,  
    age,  
    gender  
)
```

ROOM

```
ROOM(  
    room_id PK,  
    patient_id FK,  
    room_no,  
    room_type  
)
```

PRESCRIPTION

```
PRESCRIPTION(  
    prescription_id PK,  
    patient_id FK,  
    doctor_id FK,  
    medicine,  
    dosage  
)
```

BILL

```
BILL(  
    bill_id PK,  
    patient_id FK,  
    amount,  
    bill_date  
)
```

PAYMENT

```
PAYMENT(  
    payment_id PK,  
    bill_id FK,  
    payment_method,  
    payment_status  
)
```

7. Database Implementation

The database was implemented using Oracle Database

The system consists of six main tables.

7.1 Doctor Table

The Doctor table stores information about doctors working in the hospital.

```
CREATE TABLE doctor (  
    doctor_id NUMBER PRIMARY KEY,  
    doctor_name VARCHAR2(50),  
    specialization VARCHAR2(50),  
    phone VARCHAR2(15)  
);
```

Example records include doctors from different specialties such as:

- Cardiology
- Neurology
- Orthopedics
- Dermatology
- Pediatrics
- Gynecology

7.2 Patient Table

The Patient table stores patient information and identifies the doctor assigned to each patient.

```
CREATE TABLE patient (  
    patient_id NUMBER PRIMARY KEY,  
    doctor_id NUMBER,  
    patient_name VARCHAR2(50),  
    age NUMBER,  
    gender VARCHAR2(10),  
    CONSTRAINT fk_patient_doctor  
        FOREIGN KEY (doctor_id)  
        REFERENCES doctor(doctor_id)  
);
```

7.3 Room Table

The Room table stores information about room assignments.

It includes different room types such as:

- General
- Cabin

- ICU
- VIP

7.4 Prescription Table

The Prescription table stores medicine information prescribed by doctors.

It connects both the patient and doctor with the prescription.

7.5 Bill Table

The Bill table stores the financial information of patients.

It includes:

- Bill ID
- Patient ID
- Amount
- Bill date

7.6 Payment Table

The Payment table stores payment information related to hospital bills.

It contains:

- Payment ID
- Bill ID
- Payment method
- Payment status

Payment methods include examples such as:

- Cash
- Card
- Mobile Banking

8. SQL Keyword Descriptions

The following SQL keywords and concepts were used in the project.

Keyword / Concept	Description
CREATE TABLE	Creates a new database table
INSERT INTO	Adds new records to a table
SELECT	Retrieves data from tables
WHERE	Filters records based on a condition
AND	Combines multiple conditions
OR	Returns records satisfying either condition
NOT	Negates a condition
IN	Checks whether a value belongs to a specified list
NOT IN	Checks whether a value does not belong to a list
LIKE	Searches for a specific text pattern
SOME	Compares a value with at least one value returned by a subquery
ALL	Compares a value with every value returned by a subquery
EXISTS	Checks whether a subquery returns any record
GROUP BY	Groups records having similar values
HAVING	Filters grouped records
ORDER BY	Sorts query results
JOIN	Combines related records from multiple tables
INNER JOIN	Returns matching records from tables

Keyword / Concept	Description
LEFT JOIN	Returns all records from the left table and matching records from the right
RIGHT JOIN	Returns all records from the right table and matching records from the left
FULL JOIN	Returns matching and non-matching records from both tables
UNION	Combines results and removes duplicates
UNION ALL	Combines results including duplicates
INTERSECT	Returns common records
MINUS	Returns records from the first query that are absent from the second
COUNT()	Counts records
SUM()	Calculates total
AVG()	Calculates average
MAX()	Finds the maximum value
MIN()	Finds the minimum value
CREATE VIEW	Creates a virtual table based on a query
ALTER TABLE	Changes the structure of a table

9. SQL Queries & Outputs

The project contains several SQL queries designed around realistic hospital operations.

9.1 Patient and Doctor Information

SELECT

p.patient_id,
p.patient_name,
p.age,

```
p.gender,  
d.doctor_name,  
d.specialization  
FROM patient p  
JOIN doctor d  
ON p.doctor_id = d.doctor_id;
```

9.2 Patient Billing Report

```
SELECT  
p.patient_name,  
b.bill_id,  
b.amount,  
b.bill_date,  
pay.payment_method,  
pay.payment_status  
FROM patient p  
JOIN bill b  
ON p.patient_id = b.patient_id  
JOIN payment pay  
ON b.bill_id = pay.bill_id;
```

9.3 Patients with Their Doctors and Prescriptions

```
SELECT  
p.patient_name,  
d.doctor_name,  
d.specialization,
```

```
    pr.medicine,  
    pr.dosage  
FROM patient p  
JOIN doctor d  
    ON p.doctor_id = d.doctor_id  
JOIN prescription pr  
    ON p.patient_id = pr.patient_id;
```

9.4 Total Hospital Billing

```
SELECT  
    SUM(amount) AS total_hospital_bill  
FROM bill;
```

9.5 Highest Patient Bill

```
SELECT  
    p.patient_name,  
    b.amount  
FROM patient p  
JOIN bill b  
    ON p.patient_id = b.patient_id  
WHERE b.amount = (  
    SELECT MAX(amount)  
    FROM bill  
);
```

9.6 ICU Patient Report

```
SELECT  
    p.patient_id,
```

```
p.patient_name,  
r.room_no,  
r.room_type  
FROM patient p  
JOIN room r  
ON p.patient_id = r.patient_id  
WHERE r.room_type = 'ICU';
```

9.7 Pending Payment Report

```
SELECT  
p.patient_name,  
b.bill_id,  
b.amount,  
pay.payment_method,  
pay.payment_status  
FROM patient p  
JOIN bill b  
ON p.patient_id = b.patient_id  
JOIN payment pay  
ON b.bill_id = pay.bill_id  
WHERE pay.payment_status = 'Pending';
```

9.8 SELECT INTO with %TYPE

```
DECLARE  
v_patient_name patient.patient_name%TYPE;  
  
BEGIN  
  
SELECT patient_name  
INTO v_patient_name
```

```
FROM patient
```

```
WHERE patient_id = 201;
```

```
DBMS_OUTPUT.PUT_LINE('Patient Name: ' || v_patient_name);
```

```
END;
```

```
/
```

9.9 SELECT INTO with %ROWTYPE

```
DECLARE
```

```
    v_patient patient%ROWTYPE;
```

```
BEGIN
```

```
    SELECT *
```

```
    INTO v_patient
```

```
    FROM patient
```

```
    WHERE patient_id = 201;
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
    'Patient: ' || v_patient.patient_name
```

```
);
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
    'Age: ' || v_patient.age
```

```
);
```

```
END;
```

```
/
```

9.10 IF-ELSE

```
DECLARE
```

```

    v_age patient.age%TYPE;
BEGIN
    SELECT age
    INTO v_age
    FROM patient
    WHERE patient_id = 201;

    IF v_age >= 18 THEN
        DBMS_OUTPUT.PUT_LINE('Adult Patient');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Minor Patient');
    END IF;
END;
/

```

9.11 LOOP

```

DECLARE
    v_count NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE('Processing patient record: ' || v_count);

        v_count := v_count + 1;

        EXIT WHEN v_count > 5;
    END LOOP;
END;

```

/

9.12 WHILE LOOP

DECLARE

 v_count NUMBER := 1;

BEGIN

 WHILE v_count <= 5 LOOP

 DBMS_OUTPUT.PUT_LINE(

 'Checking hospital record: ' || v_count

);

 v_count := v_count + 1;

 END LOOP;

END;

/

9.13 FOR LOOP

BEGIN

 FOR rec IN (

 SELECT patient_id, patient_name

 FROM patient

 ORDER BY patient_id

) LOOP

 DBMS_OUTPUT.PUT_LINE(

 rec.patient_id || ' - ' || rec.patient_name

```
);
```

```
END LOOP;
```

```
END;
```

```
/
```

9.14 Exception Handling

```
DECLARE
```

```
    v_patient_name patient.patient_name%TYPE;
```

```
BEGIN
```

```
    SELECT patient_name
```

```
    INTO v_patient_name
```

```
    FROM patient
```

```
    WHERE patient_id = 999;
```

```
    DBMS_OUTPUT.PUT_LINE(v_patient_name);
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Patient record not found.');
```

```
    WHEN TOO_MANY_ROWS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Multiple patient records found.');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
```

```
END;
```

/

9.15 Function

A function is used for hospital billing calculation.

```
CREATE OR REPLACE FUNCTION hospital_bill(
```

```
    p_amount NUMBER
```

```
)
```

```
RETURN NUMBER
```

```
IS
```

```
    v_total NUMBER;
```

```
BEGIN
```

```
    v_total := p_amount + (p_amount * 0.05);
```

```
    RETURN v_total;
```

```
END;
```

/

Example:

```
SELECT
```

```
    bill_id,
```

```
    amount,
```

```
    hospital_bill(amount) AS total_with_service_charge
```

```
FROM bill;
```

9.16 Cursor

A cursor is used to process patient and doctor records.

```
DECLARE
```

```
    CURSOR patient_cursor IS
```

```
    SELECT
```

```

        p.patient_id,
        p.patient_name,
        d.doctor_name,
        d.specialization
FROM patient p
JOIN doctor d
    ON p.doctor_id = d.doctor_id
ORDER BY p.patient_id;
BEGIN
FOR rec IN patient_cursor LOOP

    DBMS_OUTPUT.PUT_LINE(
        'Patient ID: ' || rec.patient_id
    );
    DBMS_OUTPUT.PUT_LINE(
        'Patient: ' || rec.patient_name
    );
    DBMS_OUTPUT.PUT_LINE(
        'Doctor: ' || rec.doctor_name
    );
    DBMS_OUTPUT.PUT_LINE(
        'Specialization: ' || rec.specialization
    );
    DBMS_OUTPUT.PUT_LINE(
        '-----'
    );

```

END LOOP;

END;

/

10. Screenshots of outputs

Only selected outputs should be included in the report rather than taking screenshots of every query.

Recommended screenshots:

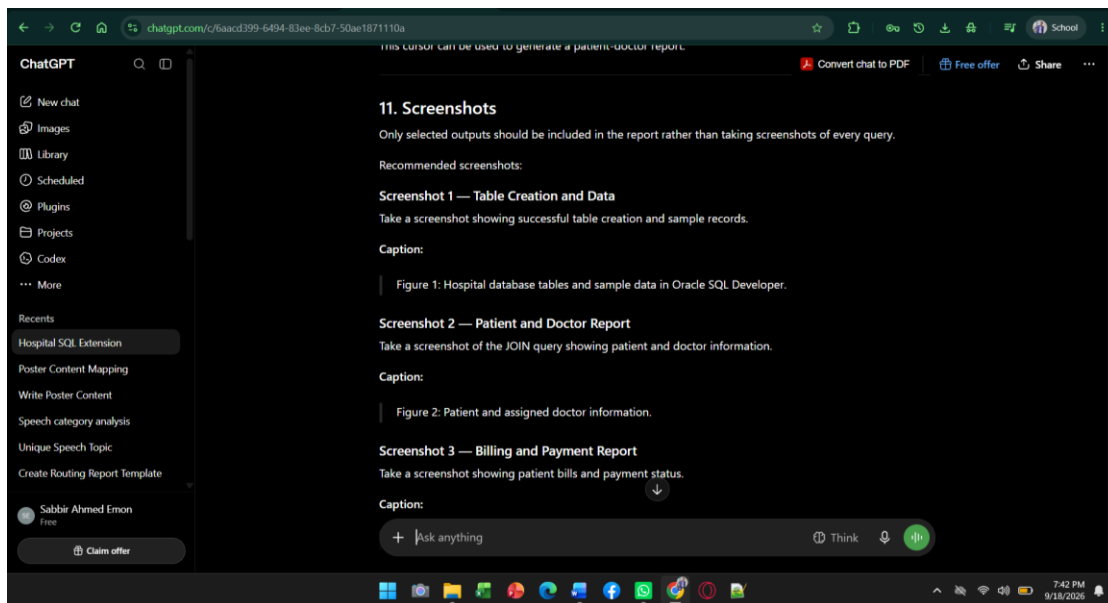


Figure 1: Hospital database tables and sample data

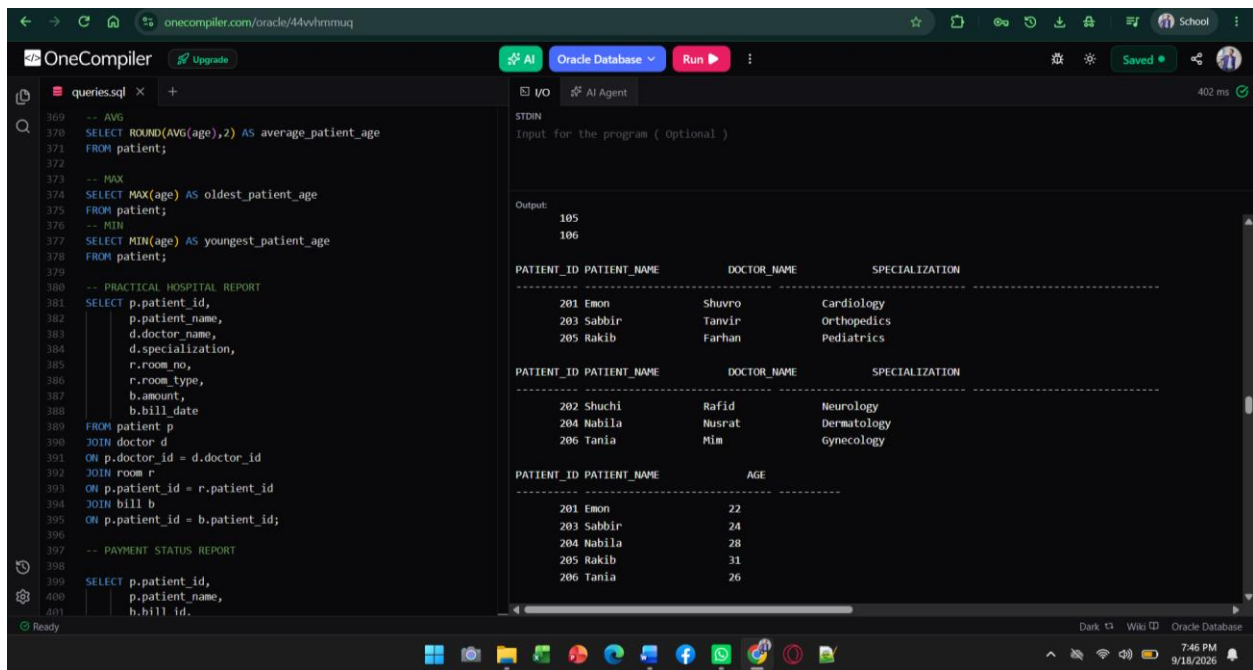


Figure 2: Patient and assigned doctor information.

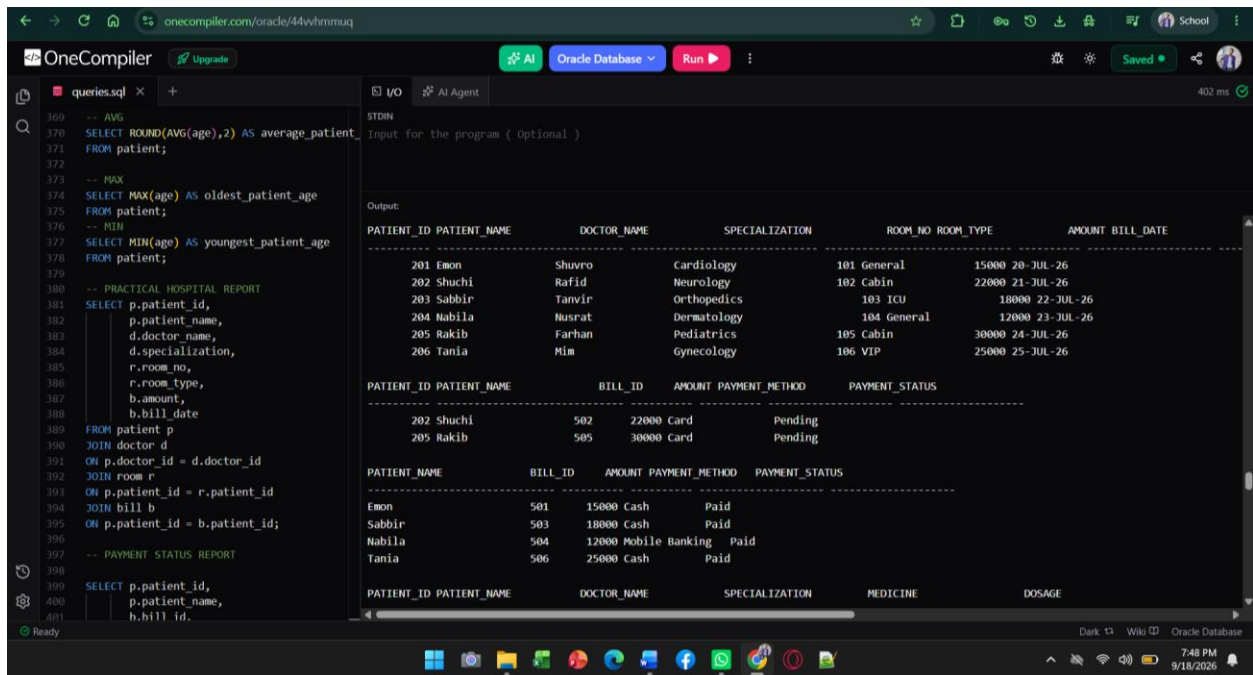


Figure 3: Hospital billing and payment report.

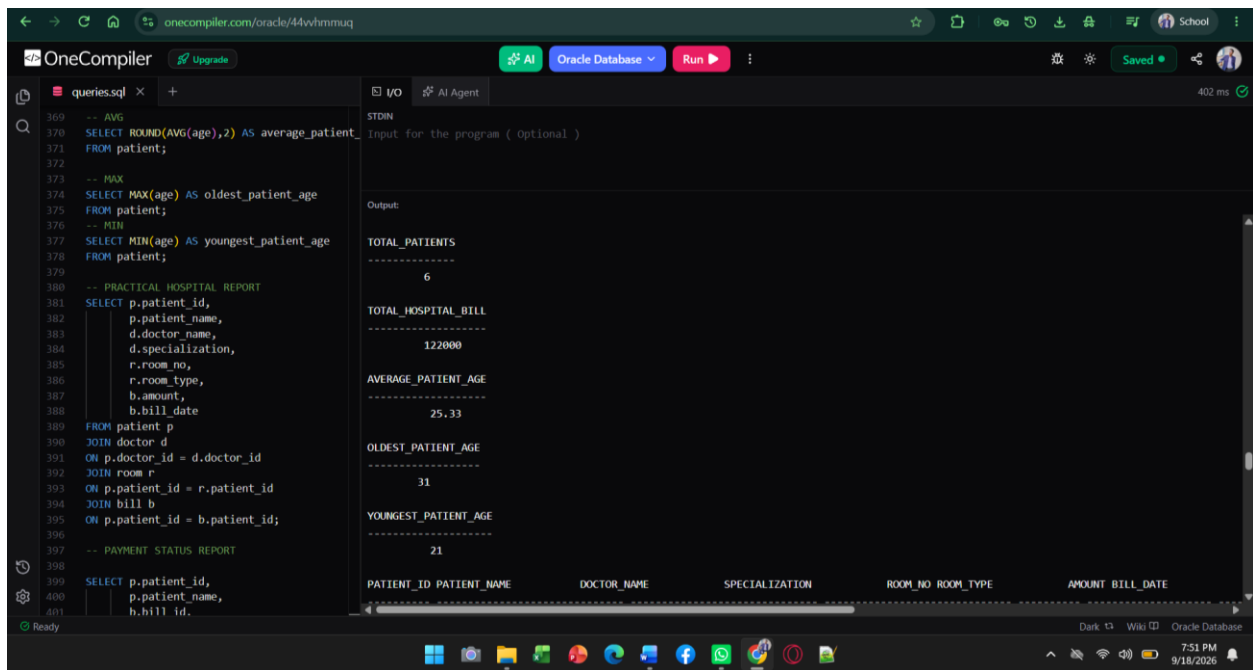


Figure 4: Hospital statistical information using aggregate functions.

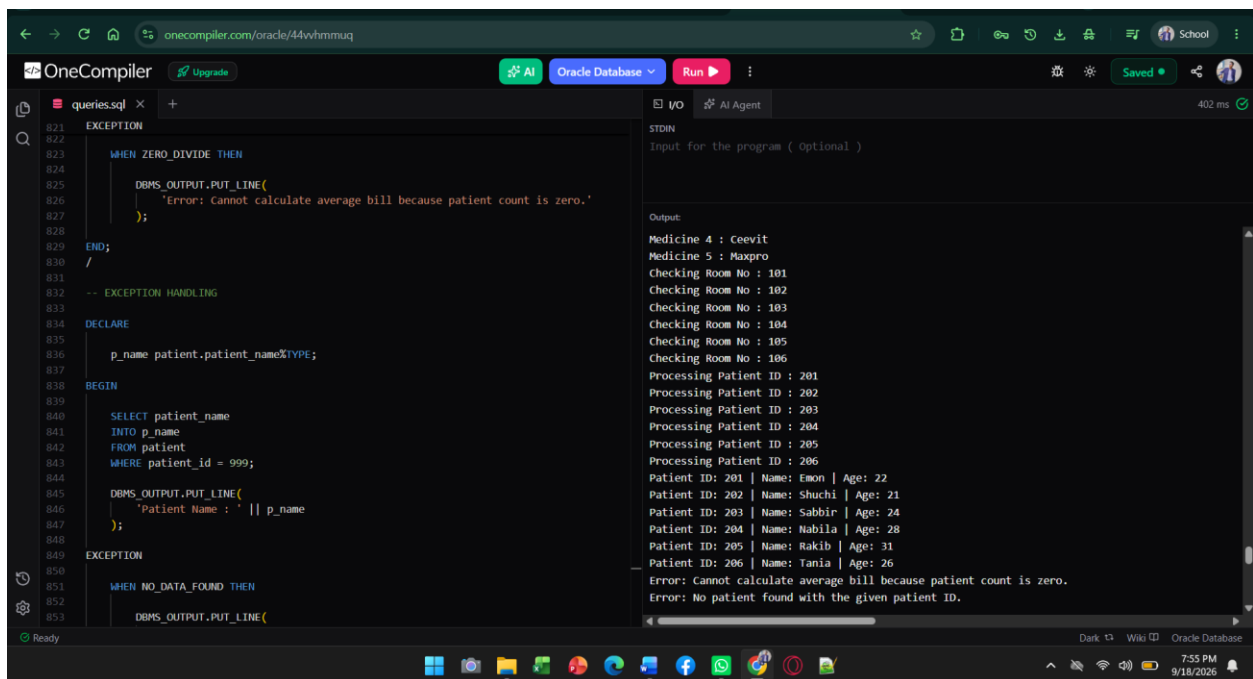


Figure 5: PL/SQL patient summary output.

```

1003 DECLARE
1004
1005     CURSOR patient_cursor IS
1006
1007         SELECT p.patient_id,
1008                p.patient_name,
1009                p.age,
1010                d.doctor_name,
1011                d.specialization
1012         FROM patient p
1013         JOIN doctor d
1014         ON p.doctor_id = d.doctor_id
1015         ORDER BY p.patient_id;
1016
1017     p_id patient.patient_id%TYPE;
1018     p_name patient.patient_name%TYPE;
1019     p_age patient.age%TYPE;
1020     d_name doctor.doctor_name%TYPE;
1021     d_specialization doctor.specialization%TYPE;
1022
1023 BEGIN
1024
1025     OPEN patient_cursor;
1026
1027     LOOP
1028
1029         FETCH patient_cursor
1030         INTO p_id,
1031              p_name,
1032              p_age,
1033              d_name,
1034              d_specialization;
1035

```

Output:

502 Shuchi	22000
Pending	
503 Sabbir	18000
Paid	
504 Nabila	12000
Paid	
505 Rakib	30000
Pending	
506 Tania	25000
Paid	

Patient ID: 201 | Patient: Emon | Age: 22 | Doctor: Shuvro | Department: Cardiology
 Patient ID: 202 | Patient: Shuchi | Age: 21 | Doctor: Rafid | Department: Neurology
 Patient ID: 203 | Patient: Sabbir | Age: 24 | Doctor: Tanvir | Department: Orthopedics
 Patient ID: 204 | Patient: Nabila | Age: 28 | Doctor: Nusrat | Department: Dermatology
 Patient ID: 205 | Patient: Rakib | Age: 31 | Doctor: Farhan | Department: Pediatrics
 Patient ID: 206 | Patient: Tania | Age: 26 | Doctor: Mim | Department: Gynecology

Figure 6: Patient-doctor information retrieved from the database view.

11. Discussion

This Hospital Management System shows how a database can be used to manage hospital information in a simple and organized way.

The system uses different tables for doctors, patients, rooms, prescriptions, bills, and payments. This helps keep the data organized and avoids storing everything in one table.

Primary keys are used to identify each record uniquely. Foreign keys are used to connect the tables. For example, the `doctor_id` in the Patient table connects a patient with their doctor.

Different SQL queries are used to find useful hospital information. Joins are used to get information from more than one table. Aggregate functions such as `COUNT`, `SUM`, `AVG`, `MAX`, and `MIN` are used to calculate different types of information.

The project also uses `UNION`, `UNION ALL`, `INTERSECT`, and `MINUS` to work with data from different queries. Subqueries using `IN`, `NOT IN`, `SOME`, `ALL`, and `EXISTS` are also used to find specific information.

PL/SQL is used to add more functionality to the database. `%TYPE` and `%ROWTYPE` are used to store database values and records. `IF-ELSE` is used for decision making, while arrays and loops are used to process multiple values. Exception handling is used to handle errors, such as when a patient record cannot be found.

Procedures, functions, and cursors are also used to perform different hospital-related tasks in a more organized way.

Overall, this project helped us understand how SQL and PL/SQL can be used together to create a simple and organized Hospital Management System.

12. Conclusion

The Hospital Management System project was developed using Oracle SQL and PL/SQL. The system stores and manages important hospital information such as doctors, patients, rooms, prescriptions, bills, and payments.

In this project, we used primary keys and foreign keys to connect the different tables. We also used different SQL concepts such as queries, joins, subqueries, set operators, aggregate functions, grouping, and views.

The project also includes PL/SQL concepts such as %TYPE, %ROWTYPE, IF-ELSE, arrays, loops, exception handling, procedures, functions, and cursors.

This project helped us understand how the database concepts we learned in the lab can be used in a real-life system. In the future, the system can be improved by adding features such as appointment management, user login, medicine management, online payment, and a graphical user interface.

13. References

You can use the following references:

1. Database Management System Lab Lecture Slides – Provided by the Course Instructor through Google Classroom, Southeast University.
2. Database Management System Lab Manual – Provided by the Course Instructor.
3. Oracle SQL and PL/SQL Documentation – Used for database implementation.

14. Appendix — Source Code

The complete source code for the Hospital Management System is provided in the appendix.

Source Code Link

GitHub:

<https://github.com/SabbirEmon2002>